

UNIVERSITY OF PISA

Computer Engineering

Internet of Things

**Autonomous Energy Management for a Remote
Environmental Monitoring Station**

Professors:

Giuseppe Anastasi
Francesca Righetti
Carlo Vallati

Student:

Pietro Gemelli
Van Khai Do

Contents

1	Introduction	1
2	Use Case	2
2.1	Scenario	2
2.2	Actors	2
2.3	Workflow	2
2.4	Requirements Driven by the Scenario	3
3	System Architecture	3
3.1	Nodes	3
3.2	Protocols	4
4	Energy Balance and Battery Autonomy	4
4.1	Balance Loop	4
4.2	Diesel as Emergency Backup	4
4.3	Autonomous Battery Protection	5
5	Stress Test and Adaptive Mechanisms	5
5.1	Network Congestion	5
5.1.1	Stress Test Result	5
5.1.2	Adaptive Mechanism	6
5.1.3	Threshold Justification and With/Without Comparison	6
5.2	Node Failure	7
5.2.1	Stress Test Result	7
5.2.2	Adaptive Mechanism	8
6	Machine Learning	8
7	Limitations and Future Works	8
7.1	Limitations	8
7.2	Future Works	9
8	Conclusion	9

1. Introduction

Autonomous, self-sufficient power systems are essential wherever continuous grid access is unavailable and human supervision is rare. This project designs and implements an IoT-based energy management system for exactly this setting, combining renewable production, battery storage with autonomous protection, a dispatchable diesel backup, and a network layer that adapts its own behaviour under stress ,built on Contiki-NG/Cooja, CoAP, MQTT, InfluxDB, and Grafana.

The objectives are: (1) prioritise renewable production and use the battery to buffer short-term surplus/deficit; (2) protect the battery from unsafe operation using on-device logic, independent of the cloud; (3) dispatch a diesel generator only as a last resort; (4) keep telemetry reliable under network stress by

adapting the publish rate autonomously.

2. Use Case

2.1 Scenario

A research organisation operates an unmanned environmental monitoring station, collecting meteorological, hydrological, or ecological data, in a location with no grid connection and an unreliable low-bandwidth radio/satellite backhaul. The station must run for months without on-site maintenance, visited only occasionally by a field technician for physical upkeep. Any power interruption or prolonged loss of connectivity risks unrecoverable gaps in scientific data, since a missed measurement window (a storm event, a seasonal transition) cannot be retroactively recovered.

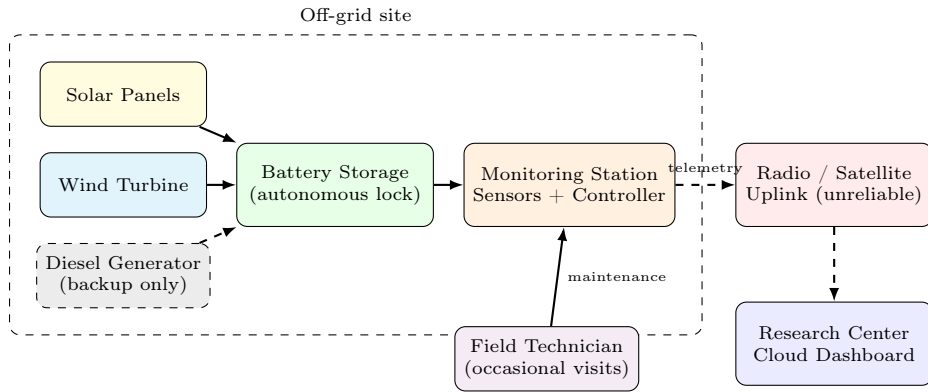


Figure 2.1: Use case scenario: an off-grid monitoring station powered by renewables and diesel backup, reporting to a remote research center over an unreliable uplink, with only occasional human intervention.

2.2 Actors

- **Research organisation:** own the station, consumes the collected data, and defines acceptable data-loss tolerance.
- **Field technician:** visits the site occasionally (e.g. seasonally) for physical maintenance — not available for real-time intervention.
- **The station itself:** must make short-term operational decisions autonomously, since neither actor above is continuously present or continuously reachable.

2.3 Workflow

The station's operation is a continuous cycle, with no assumption of reliable connectivity to the research center at any step:

1. Solar and wind continuously feed the battery; the battery buffers any short-term mismatch between production and the station's own load.
2. The battery monitors its own state of charge and refuses further charging once a safe threshold is exceeded — a decision made independently of whether the uplink is currently working.
3. If renewable production and battery reserve are both insufficient, the station dispatches the diesel generator as a last resort, without waiting for confirmation from the research center.
4. Sensor data is transmitted over the uplink when available; if the uplink shows signs of congestion (rising delay), the station reduces its own transmission rate rather than continuing to flood a struggling link.
5. All telemetry, energy state, and dispatch events are logged locally and forwarded to the research center's dashboard whenever the uplink allows.

2.4 Requirements Driven by the Scenario

- **Energy autonomy:** renewable production must be prioritised, with the battery absorbing short-term imbalance and diesel used only when the battery itself is critically low , minimising both fuel resupply trips and the risk of a dead battery.
- **On-device protection:** since no operator is present, the battery must protect itself from unsafe charge levels without waiting for a cloud decision that may never arrive if connectivity is down.
- **Resilient telemetry:** given the unreliable uplink, the system must detect degraded delivery and reduce its own transmission rate rather than flooding an already-struggling link.

3. System Architecture

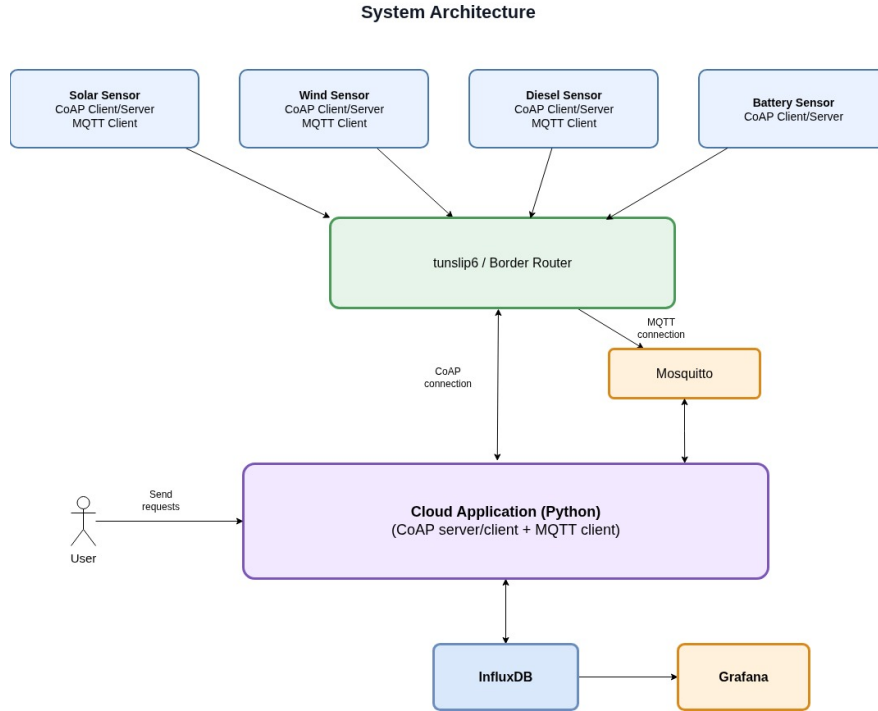


Figure 3.1: System architecture.

3.1 Nodes

Producer nodes (solar, wind, diesel) share one firmware source (`coap-mqtt-sensor.c`), differentiated by the `NODE_TYPE` macro.

- **Resources exposed:** `/actuators/leds`, `/actuators/status`, `/test/push` (observed by the cloud).
- **Function:** samples simulated voltage/current, publishes MQTT telemetry at a configurable interval, exposes remote LED and on/off control.
- **Observations:** producers do not observe other nodes.

Each registers once at startup (CoAP POST `/register`) after RPL connectivity is established; the cloud then subscribes back via CoAP Observe on `/test/push`.

The **diesel node** differs in one respect: it starts **OFF** by default and, once commanded on, undergoes a **40-second warm-up** (`status = "STARTING"`, `v = i = 0`) before producing power , modelling the mechanical spin-up of a real generator, unlike solar/wind which respond instantly.

Battery node (`coap-mqtt-battery-sensor.c`):

- **Resources exposed:** `/actuators/battery` (GET/PUT charge delta), `/actuators/battery/override`.

- **Function:** maintains `charged_capacity/max_capacity`, applies charge/discharge deltas, autonomously enforces the charge lock (Chapter 4).
- **Observations:** none. Unlike producers, it runs no MQTT loop, all interaction is CoAP, driven by explicit commands rather than periodic self-sampling.

Border router: bridges the simulated 6LoWPAN/RPL network to the host via `tunslip6`.

Cloud application: handles registration, MQTT ingestion, InfluxDB persistence, the energy balance loop, and network stress adaptive mechanisms (Chapter 4); exposes `/register`, `/unregister`, `/telemetry`, `/actuators/leds`, `/actuators/status` as forwarding endpoints.

3.2 Protocols

CoAP for registration and actuator control (sparse, request/response, needs explicit confirmation e.g. a battery rejecting a charge command). **MQTT** for telemetry streaming (frequent, one-to-many, suited to a publish/subscribe broker rather than per-node polling).

4. Energy Balance and Battery Autonomy

4.1 Balance Loop

Every `BALANCE_INTERVAL_S` (5s), the cloud aggregates renewable producer power, generates a synthetic load, reads the battery's state of charge (SoC), and computes `total_surplus = renewable_w + diesel_w - load_w`, sending a clamped charge (positive surplus) or discharge (negative surplus) command to the battery.

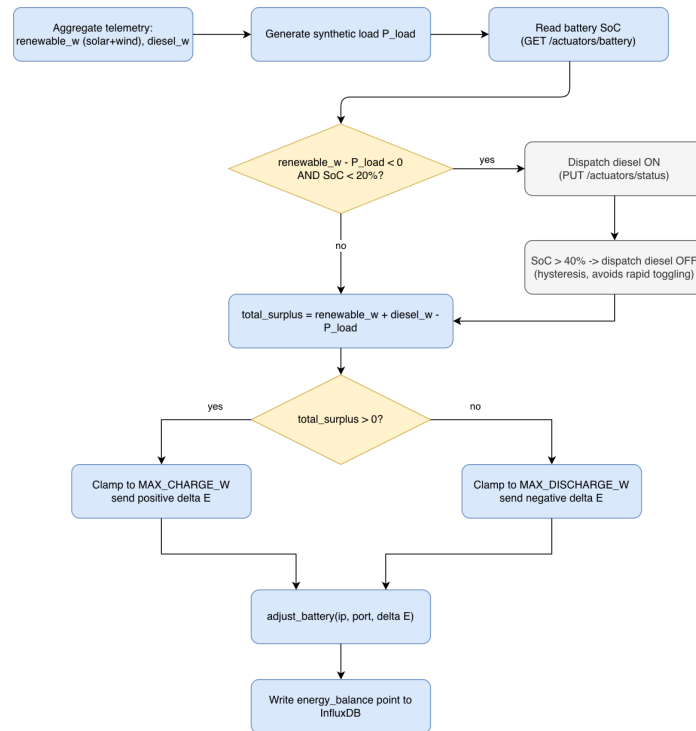


Figure 4.1: Energy balance decision loop, including diesel dispatch.

4.2 Diesel as Emergency Backup

Diesel is dispatched only when renewables cannot cover the load *and* the battery is critically low (SoC < 20%), and stopped once SoC recovers past 40% (hysteresis avoids rapid on/off cycling). This mirrors the scenario's requirement to minimise diesel use, and therefore fuel resupply trips to the station.

4.3 Autonomous Battery Protection

The battery firmware refuses further charge commands once $\text{SoC} \geq 95\%$ (CoAP 4.03 Forbidden), resuming acceptance only below 75%, a decision made entirely on-device, without depending on the cloud or connectivity to it, consistent with the scenario's unattended, uplink-unreliable setting. A manual override resource allows the cloud to bypass the lock when explicitly needed.

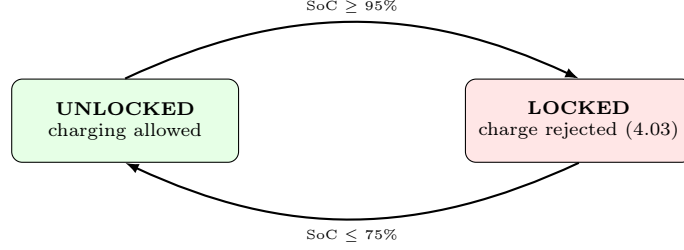


Figure 4.2: Battery charge-lock hysteresis.

5. Stress Test and Adaptive Mechanisms

5.1 Network Congestion

5.1.1 Stress Test Result

Tested on Cooja and physical dongles, publish periods 5000/2000/1000/800 ms, 2-minute windows, three node types.

Accepted loss tolerance: a packet delivery ratio of at least 80% is considered acceptable during stress conditions. Therefore, up to 20% packet loss is tolerated and attributed to network and hardware constraints.

Period (ms)	PDR (%) Cooja			PDR (%) Dongle		
	Wind	Solar	Diesel	Wind	Solar	Diesel
5000	100.0	100.0	100.0	100.0	100.0	100.0
2000	100.0	100.0	100.0	100.0	100.0	100.0
1000	91.2	100.0	96.8	100.0	100.0	100.0
800	79.0	88.8	79.7	84.6	80.3	80.8

Table 5.1: PDR by publish period.

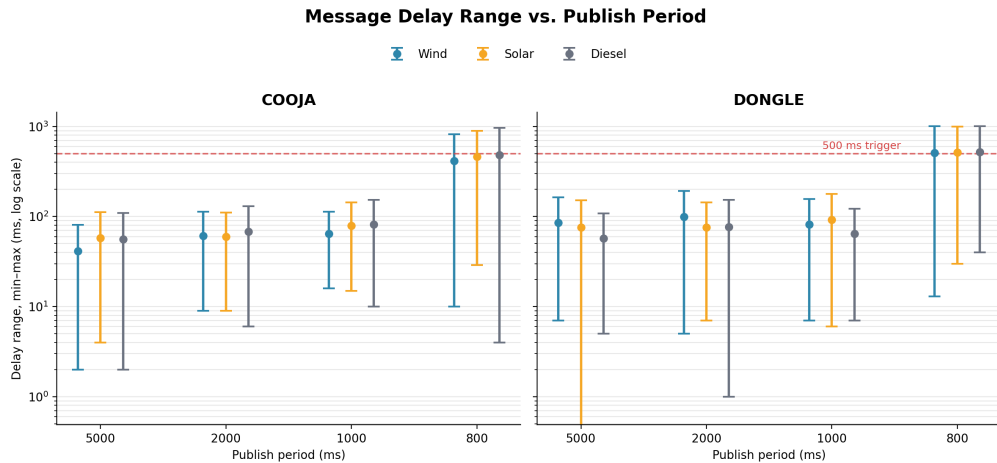


Figure 5.1: Delay range vs. publish period. The 500 ms trigger is crossed only at 800 ms, on both platforms, with no borderline cases.

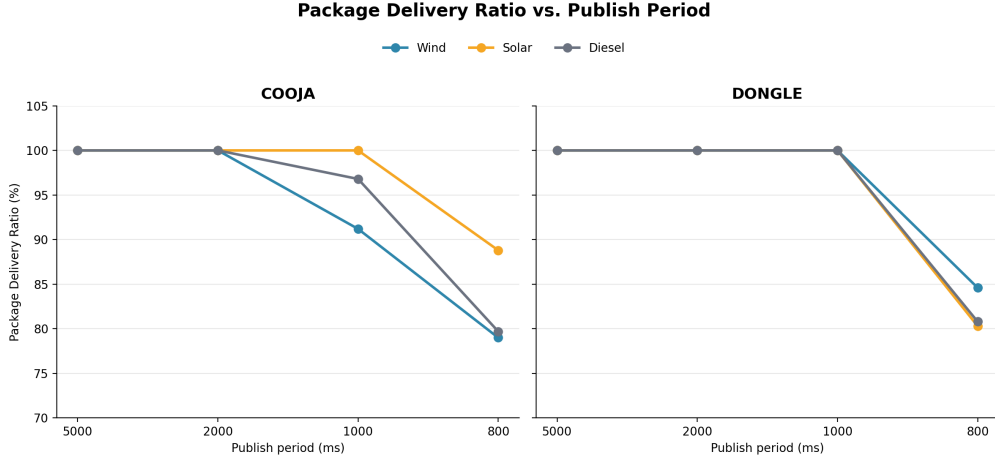


Figure 5.2: PDR vs. publish period, shown for reference against the 20% accepted loss tolerance.

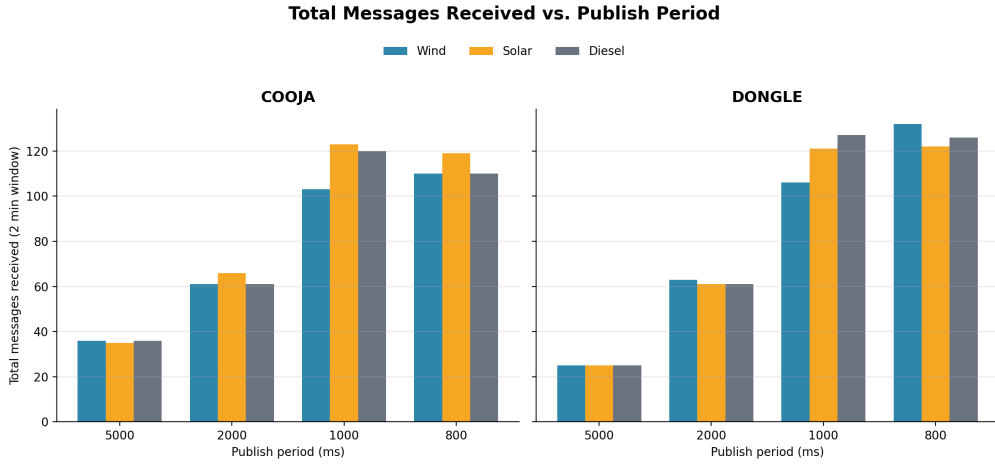


Figure 5.3: Total messages received per 2-minute window, by publish period and node type.

5.1.2 Adaptive Mechanism

The system implements an adaptive congestion-control mechanism based on the observed delay of MQTT telemetry messages.

- **Type:** adaptive MQTT sampling-rate control based on network conditions.
- **Trigger:** for each received MQTT message, the cloud estimates the message delay and maintains a moving average over the last five samples for each node. A node is marked as congested when: $average_delay > 400$ ms
- **Response:** when congestion is detected, the cloud sends a CoAP command to the affected node: PUT /actuators/sampling with the new MQTT publishing interval in the request body.
- **Congested interval:** in the current implementation, the cloud sets the congested publishing interval to: 5000 ms
- **Recovery:** when the average delay returns below the congestion threshold, the cloud restores the node's original baseline publishing interval. The baseline interval is learned using: GET /actuators/sampling

5.1.3 Threshold Justification and With/Without Comparison

Delay stays under 200 ms at 1000 ms and above; at 800 ms it reaches 818–1006 ms on both platforms, an unambiguous separation, unlike a PDR-based trigger (Cooja/wind measured a borderline 100% PDR at 1000 ms in a single run).

- **Without adaptation:** with a publishing interval of 800 ms, the network becomes overloaded. The delay increases significantly, reaching up to approximately 1006 ms, and the MQTT connection eventually disconnects.
- **With adaptation:** starting from the same 800 ms interval, the cloud detects congestion when the average delay of the last five packets exceeds 400 ms. The node publishing interval is then increased. The system remains stable and the MQTT connection does not break, although the average delay remains around 600 ms.

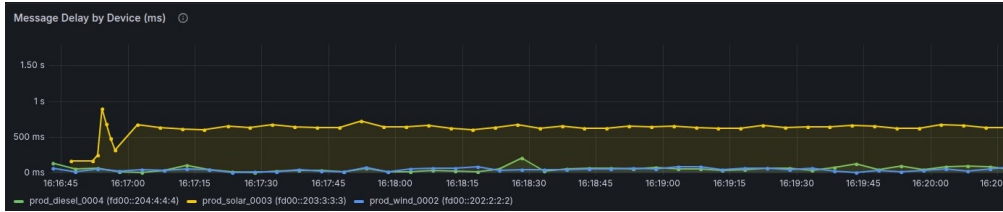


Figure 5.4: Network Congestion with Adaptive Mechanism.

The specific threshold values are appropriate for this simulated deployment, where readings carry no real operational consequence; in an actual deployment the safety margin should reflect the application's tolerance for missing or delayed data, which varies by domain. Thresholds are therefore exposed as configurable parameters rather than fixed constants.

5.2 Node Failure

5.2.1 Stress Test Result

The node failure test was performed by disabling the solar node `prod_solar_0003` while the system was running. The relevant execution logs are reported below using terminal-style log blocks.

Before the failure, all four nodes were successfully registered in the cloud registry: battery, solar, wind, and diesel.

```
2026-07-17 16:33:27,276 - INFO - [Registry] Node 'prod_battery_0005' added IP=fd00::205:5:5:5 port=5683
total=1
2026-07-17 16:33:30,613 - INFO - [Registry] Node 'prod_solar_0003' added IP=fd00::203:3:3:3 port=5683 total
=2
2026-07-17 16:33:37,877 - INFO - [Registry] Node 'prod_wind_0002' added IP=fd00::202:2:2:2 port=5683 total=3
2026-07-17 16:33:39,136 - INFO - [Registry] Node 'prod_diesel_0004' added IP=fd00::204:4:4:4 port=5683 total
=4
```

The heartbeat logs before the failure show that all registered nodes were reachable.

```
2026-07-17 16:35:48,628 - INFO - [CoAP Client] Heartbeat OK for 'prod_battery_0005' (2.05 Content)
2026-07-17 16:35:48,713 - INFO - [CoAP Client] Heartbeat OK for 'prod_solar_0003' (2.05 Content)
2026-07-17 16:35:48,804 - INFO - [CoAP Client] Heartbeat OK for 'prod_wind_0002' (2.05 Content)
2026-07-17 16:35:48,912 - INFO - [CoAP Client] Heartbeat OK for 'prod_diesel_0004' (2.05 Content)
```

Before the node failure, the energy balance was calculated using all available nodes. The renewable production was approximately between 922 W and 937.5 W.

```
2026-07-17 16:35:43,091 - INFO - [EnergyBalance] renewable=937.5W diesel=1982.7W load=1739.8W soc=50% delta
=0.00164kWh diesel_dispatched=False
2026-07-17 16:35:48,373 - INFO - [EnergyBalance] renewable=922.1W diesel=1270.5W load=791.0W soc=50% delta
=0.00195kWh diesel_dispatched=False
```

At runtime, the solar node became unreachable. The cloud detected the failure through the CoAP heartbeat mechanism and removed the failed node from the active registry.

```
2026-07-17 16:36:02,028 - WARNING - [CoAP Client] Heartbeat probe failed for 'prod_solar_0003' at coap://[fd00
::203:3:3:3]:5683/actuators/status:
2026-07-17 16:36:02,028 - WARNING - [Failure] Node 'prod_solar_0003' did not respond to CoAP heartbeat.
Removing from registry.
2026-07-17 16:36:02,028 - INFO - [Registry] Node 'prod_solar_0003' removed.
```

After the solar node was removed, the heartbeat mechanism continued monitoring the remaining active nodes. The logs show that the battery, wind, and diesel nodes were still reachable.

```

2026-07-17 16:36:14,650 - INFO - [CoAP Client] Heartbeat OK for 'prod_battery_0005' (2.05 Content)
2026-07-17 16:36:14,744 - INFO - [CoAP Client] Heartbeat OK for 'prod_wind_0002' (2.05 Content)
2026-07-17 16:36:14,840 - INFO - [CoAP Client] Heartbeat OK for 'prod_diesel_0004' (2.05 Content)

```

The energy balance also continued operating after the failure. However, the renewable production decreased to approximately 369–412 W, showing that the solar node contribution was no longer included in the calculation.

```

2026-07-17 16:36:04,020 - INFO - [EnergyBalance] renewable=369.6W diesel=1676.1W load=1542.6W soc=50% delta
=0.00070kWh diesel_dispatched=False
2026-07-17 16:36:09,235 - INFO - [EnergyBalance] renewable=412.4W diesel=2061.7W load=1018.1W soc=50% delta
=0.00202kWh diesel_dispatched=False

```

The result confirms that the system correctly detected the failed solar node, removed it from the registry, ignored its stale telemetry, and continued operating with the remaining nodes. The reduction in renewable production after the failure demonstrates the impact of the missing node on the overall energy balance.

5.2.2 Adaptive Mechanism

In the implemented system, each node first registers itself with the cloud using CoAP. The cloud stores the node information in an in-memory registry, including the node identifier, IPv6 address, port, type, and protocol. Once registered, the cloud periodically checks whether the node is still alive.

- **Failure trigger:** a node is considered faulty when it no longer responds to the cloud heartbeat probe.
- **Detection mechanism:** the cloud sends a CoAP heartbeat request every 10 seconds to each registered node.
 - Battery nodes are checked using: `GET /actuators/battery`
 - solar, wind, and diesel nodes are checked using: `GET /actuators/status`
- **Timeout:** if the node does not reply within 3 seconds, the heartbeat is considered failed.
- **Cloud response:** when a heartbeat fails, the cloud logs a failure event and removes the node from the active registry.
- **Telemetry response:** once the node is removed from the registry, its previous readings are no longer used by the energy balancing algorithm.

6. Machine Learning

A lightweight on-device classifiers are planned, using decision trees exported to C (`m2cgen`), given the memory constraints of simulated nodes:

- **Generic voltage/current fault classifier,** deployed on the shared producer firmware (covering solar, wind, and diesel through one code path), trained on the Kaggle Dataset *Electrical Fault Detection and Classification* dataset. A single-phase feature pair (V_a , I_a) is used to match the firmware’s single-phase v/i telemetry.
- **Autonomous action:** on fault detection, a node can locally set its own `device_on` flag to false, without cloud involvement.

7. Limitations and Future Works

7.1 Limitations

- **Non-persistent node registry:** restarting the cloud application without also restarting the Cooja simulation loses all registrations, since registration is a one-shot firmware behaviour with no periodic retry.
- **Duplicate Observe subscriptions:** the registry does not check for existing registrations before triggering a new CoAP Observe subscription, producing orphaned notifications on re-registration.
- **Synthetic load realism:** the consumer load is pseudo-random noise without temporal correlation.

7.2 Future Works

- Deploy the application in real-world scenarios, beyond the current Cooja/dongle simulation.
- Improve the ML model using a more specific dataset, tailored to our own collected metrics.
- Implement a smarter battery cut-off and more comprehensive BMS management.
- Introduce a data layer for transmitting the actual scientific measurements collected by the station, on top of the existing energy-management telemetry.

8. Conclusion

The system demonstrates autonomous operation suited to an unattended, connectivity-constrained monitoring station renewable-first energy balancing, on-device battery protection, diesel as last resort, and a network layer designed to degrade gracefully rather than fail under stress.